

Hyperon-Experimental as PyMeTTa / metta_magic

There are two main mechanisms for importing Python objects into MeTTa

1. Creation of H-E extension with registration of grounded atoms

```
# sum_squares.py
import jax.numpy as jnp
from hyperon import *
from hyperon.ext import register_atoms

def sum_squares(xs):
    arr = jnp.array(xs)
    return int(jnp.sum(arr*arr))

@register_atoms
def def_atoms():
    return {
        'sum_squares': OperationAtom('sum_squares', sum_squares)
    }
```

```
# call_sum.metta
!(import! &self sum_squares)
!(sum_squares (py-list (1 2 3))) ; 14
```

So, calling `sum_squares` from MeTTa is that simple. It can be somewhat simplified further. The idea behind `register_atoms` is that it is more universal. One can create regular expressions for parsing code into grounded objects, such as strings, numbers, etc. But for this very simple case, when we have just a grounded function, which is directly wrapped into the grounded atom represented by the same name and parsed from the same textual representation, we could indeed introduce `@grounded` decorator to the function itself. It is easily doable.

2. Accessing Python objects via py-atom

sum_squares.py

```
import jax.numpy as jnp

def sum_squares(xs):
    arr = jnp.array(xs)
    return int(jnp.sum(arr*arr))
```

call_sum.metta

```
!((py-atom sum_squares.sum_squares) (py-list (1 2 3))) ; 14
```

sum_squares.sum_squares means that we take sum_squares object/function from sum_squares.py file. That's all. This just works.

3. Accessing Python from MeTTa from Python

This can be done for both versions above, but let's put everything in one script, so we will not register an extension.

```
import jax.numpy as jnp
from hyperon import *

# let's name the function differently from the file to avoid
# confusion
def sum_sq(xs):
    arr = jnp.array(xs)
    return int(jnp.sum(arr*arr))

metta = MeTTa()

# this is just a syntactic sugar to avoid py-atom in calls
# it could be loaded from file
metta.run('''
(= (sum_squares $xs)
```

```

    ((py-atom sum_sq) $xs))
'''

print(metta.run("! (sum_squares (py-list (1 2 3)))")) # 14
print(metta.evaluate_atom(E(S('sum_squares'), ValueAtom([1, 2,
3])))) #14

```

Calling MeTTa functions from Python can be made a little bit more convenient for particular cases.

4. Accessing Python from MeTTa from Python with atom creation

Instead of importing a Python extension from MeTTa or accessing Python functions via py-dot, one can also create atoms directly in the Python script itself and additionally use `register_atom` to parse them from textual code if needed.

```

import jax.numpy as jnp
from hyperon import *

def sum_square(xs):
    arr = jnp.array(xs)
    return int(jnp.sum(arr*arr))

metta = MeTTa()
SumSquaresAtom = OperationAtom("sum_squares", sum_square)
# We can address this atom directly
print(metta.evaluate_atom(E(SumSquaresAtom, ValueAtom([1, 2,
3])))) #14

# But we need to register a token to get this atom from parsing
metta.register_atom("sum_squares", SumSquaresAtom)
print(metta.run("! (sum_squares (py-list (1 2 3)))")) # 14

```